

Стандарт разработки на JavaScript

При разработке на JavaScript следует руководствоваться теми же правилами, которые описаны в [стандарте TypeScript](#) за исключением следующего:

- Разрешено использовать только синтаксис, соответствующий стандарту ES5. Например, нельзя использовать стрелочные функции или шаблонные строки.
- Структурный отступ — три пробела.
- Создавая ссылку на `this`, используйте `self`.



Использовать синтаксис, соответствующий стандарту ES5

Нельзя использовать стрелочные функции или шаблонные строки.

```
1 // Плохо
2 function() {
3     var that = this;
4     return function() {
5         console.log(that);
6     };
7 }

1 // Хорошо
2 function() {
3     var self = this;
4     return function() {
5         console.log(self);
6     };
7 }
```

Поднятие переменных или [hoisting](#)

- Объявление переменных поднимает переменные в начало области видимости, но не их значение.

```
1 // Мы знаем, что это работать не будет (при условии,
2 // что нет глобальной переменной notDefined).
3 function example() {
4     // => Выбрасывает код с ошибкой ReferenceError.
5     console.log(notDefined);
6 }

1 // Декларирование переменной после ссылки на нее
2 // будет работать из-за поднятия переменной.
3 // Обратите внимание: присвоенное переменной
4 // значение 'true' не будет поднято.
5 function example() {
6     // => undefined
7     console.log(declaredButNotAssigned);
8     var declaredButNotAssigned = true;
9 }

1 // Интерпретатор переносит объявление переменной
2 // к верху области видимости.
3 // Это означает, что наш пример может быть переписан как:
```

```

4 function example() {
5     var declaredButNotAssigned;
6     // => undefined
7     console.log(declaredButNotAssigned);
8     declaredButNotAssigned = true;
9 }

```

- Объявление анонимной функции поднимает в начало области видимости ее переменную, но не ее значение.

```

1 function example() {
2     // => undefined
3     console.log(anonymous);
4
5     // => TypeError anonymous is not a function
6     anonymous();
7
8     var anonymous = function() {
9         console.log('анонимная функция');
10    };
11 }

```

- Объявление именованной функции поднимает в начало области видимости переменную, не ее значение.

```

1 function example() {
2     // => undefined
3     console.log(named);
4
5     // => TypeError named is not a function
6     named();
7
8     // => ReferenceError superPower is not defined
9     superPower();
10
11    var named = function superPower() {
12        console.log('Flying');
13    };
14
15
16    // Тоже справедливо, когда имя функции совпадает с именем
    // переменной.
17    function example() {
18        // => undefined
19        console.log(named);
20
21        // => TypeError named is not a function
22        named();
23
24        var named = function named() {
25            console.log('named');
26        };
27    }
28 }

```

- Объявление функции поднимает в начало текущей области видимости и имя, и свое значение.

```

1 function example() {
2     // => Взлетаю!
3     superPower();
4

```

```

5     function superPower() {
6         console.log('Взлетаю!');
7     }
8 }

```

Более подробно можно прочитать в статье [JavaScript Scoping & Hoisting](#) от [Ben Cherry](#)

Соглашения об именовании

Рекомендация: присваивайте имена функциям — это повышает читаемость сообщений об ошибках кода.

```

1 // Плохо
2 var log = function(msg) {
3     console.log(msg);
4 };

1 // Хорошо
2 var log = function log(msg) {
3     console.log(msg);
4 };

```

Подключение статических зависимостей

Для подключения статических зависимостей используйте функцию [define со списком зависимостей](#).

```

1 define('My/Component', ['UI/Base', 'Types/entity'], function(Base,
2     entity) {
3     var Control = Base.Control;
4     var ReactiveObject = entity.ReactiveObject;
5 });

```

Подключение динамических зависимостей

- Для подключения динамических зависимостей используйте зависимость от служебного модуля [require](#), которая вернет вам функцию загрузки.
- Обязательно обрабатывайте ошибки подключения! Как минимум их следует залогировать.

```

1 // Плохо: не обработаны ошибки
2 define('My/Component', ['require'], function(require) {
3     ...
4     require(['Controls/buttons'], function(buttons) {
5         // Work with buttons
6     });
7 });

1 // Хорошо
2 define('My/Component', ['require', 'Env/Env'], function(require, Env)
3 {
4     ...
5     require(['Controls/buttons'], function(buttons) {
6         // Work with buttons
7     }, function(err) {
8         Env.IoC.resolve('ILogger').error('My/Component', err);
9     });
10 });

```

- Не используйте глобальные переменные `require` и `requirejs`.
- Не используйте устаревший модуль `Core/moduleStubs`.

```

1 // Плохо: использование глобальной переменной require
2 define('My/Component', function() {
3     ...
4     require(['Controls/buttons'], function(buttons) {
5         // Work with buttons
6     });
7 });
1 // Плохо: использование глобальной переменной requirejs
2 define('My/Component', function() {
3     ...
4     requirejs(['Controls/buttons'], function(buttons) {
5         // Work with buttons
6     });
7 });
1 // Плохо: использование Core/moduleStubs
2 define('My/Component', ['Core/moduleStubs'], function(moduleStubs) {
3     ...
4     moduleStubs.require(['Controls/buttons'], function(result) {
5         // Work with result
6     });
7 });

```

Модули

- Файл должен быть именован с [UpperCamelCase](#), располагаться в папке с таким же именем и совпадать с именем экспортируемой переменной.
- Всегда объявляйте 'use strict'; в начале модуля.

```

1 // FancyInput/Collector.js
2
3 define('FancyInput/Collector', function() {
4     'use strict';
5
6     function Collector(options) {
7         this.options = options || {};
8     }
9
10    Collector.noConflict = function () {
11        // ...
12    };
13
14    return Collector;
15 });

```

jQuery

- Для подключения библиотеки jQuery используйте зависимость от модуля 'jquery'.

```

1 // Плохо
2 define('My/Awesome/Class', function() {
3     var sidebar = $('.sidebar');
4 });
5
6 // Хорошо
7 define('My/Awesome/Class', ['!browser?jquery'], function(jquery) {

```

```

8   var $sidebar = jquery('.sidebar');
9   });

```

- Для jQuery-переменных используйте префикс \$.

```

1   // Плохо
2   var sidebar = $('.sidebar');
3
4   // Хорошо
5   var $sidebar = $('.sidebar');

```

- Кэшируйте jQuery-запросы.

```

1   // Плохо
2   function setSidebar() {
3       $('.sidebar').hide();
4
5       ...
6
7       $('.sidebar').css({
8           'background-color': 'pink'
9       });
10  }
11
12  // Хорошо
13  function setSidebar() {
14      var $sidebar = $('.sidebar');
15      $sidebar.hide();
16
17      ...
18
19      $sidebar.css({
20          'background-color': 'pink'
21      });
22  }

```

- Для DOM-запросов используйте классический каскадный CSS-синтаксис \$('.sidebar ul') или родитель > потомок \$('.sidebar > ul'). [jsPerf](#)
- Используйте find с запросами области видимости jQuery-объектов.

```

1   // Плохо
2   $('.sidebar', 'ul').hide();
3
4   // Плохо
5   $('.sidebar').find('ul').hide();
6
7   // Хорошо
8   $('.sidebar ul').hide();
9
10  // Хорошо
11  $('.sidebar > ul').hide();
12
13  // Хорошо (slower)
14  $sidebar.find('ul');
15
16  // Хорошо (faster)
17  $($sidebar[0]).find('ul');

```

Совместимость ECMAScript 5

-
- Обращайтесь к [таблице совместимости](#) с ES5 от [Kangax](#).

Производительность

- [On Layout & Web Performance](#)
- [String vs Array Concat](#)
- [Try/Catch Cost In a Loop](#)
- [Bang Function](#)
- [jQuery Find vs Context, Selector](#)
- [innerHTML vs textContent for script text](#)
- [Long String Concatenation](#)

Ресурсы

Прочитайте это

- [Annotated ECMAScript 5.1](#)
- [Ревью кода на JS](#)

Другие [руководства стиля](#)

- [Google JavaScript Style Guide](#)
- [jQuery Core Style Guidelines](#)
- [Principles of Writing Consistent, Idiomatic JavaScript](#)

Другие стили

- [Naming this in nested functions](#) - Christian Johansen
- [Conditional Callbacks](#)

Дальнейшее прочтение

- [Understanding JavaScript Closures](#) - Angus Croll

Литература

- [JavaScript: The Good Parts](#) - Douglas Crockford
- [JavaScript Patterns](#) - Stoyan Stefanov
- [Pro JavaScript Design Patterns](#) - Ross Harmes and Dustin Diaz
- [High Performance Web Sites: Essential Knowledge for Front-End Engineers](#) - Steve Souders
- [Maintainable JavaScript](#) - Nicholas C. Zakas
- [JavaScript Web Applications](#) - Alex MacCaw
- [Pro JavaScript Techniques](#) - John Resig
- [Smashing Node.js: JavaScript Everywhere](#) - Guillermo Rauch

Блоги

- [DailyJS](#)
- [JavaScript Weekly](#)
- [JavaScript, JavaScript...](#)
- [Bocoup Weblog](#)
- [Adequately Good](#)
- [NCZOnline](#)
- [Perfection Kills](#)
- [Ben Alman](#)
- [Dmitry Baranovskiy](#)
- [Dustin Diaz](#)
- [nettuts](#)

- [eslinttrc](#)